



Plataforma Backend de Temática Pokémon

Memoria Técnica del Proyecto

Node.js · Express · Sequelize ORM · PostgreSQL · POSTMAN

Autor	Módulo responsable
Asier Gonzales	User · Auth · Relaciones entre modelos
Darío Arenaza	Pokemon · Documentación técnica
Luis Alonso	Product · Vistas (Pug)
Eli Fernández	Team · TeamPokemon - Diagramas

Desarrollo Backend · Abril 2026

1. Introducción

POKEback es el backend de una plataforma de temática Pokémon desarrollada como proyecto grupal. Su propósito es ofrecer una API REST completa que gestione usuarios, Pokémon, productos de tienda y equipos, sentando las bases para futuras mecánicas de juego PvE (jugador contra entorno) y PvP (jugador contra jugador).

El proyecto nace con la filosofía de:

"Menos es más, si está bien hecho."

Esto implica priorizar la calidad del código, la coherencia del diseño y la seguridad frente a la cantidad de funcionalidades. Cada decisión técnica ha sido tomada buscando la claridad, la mantenibilidad y las buenas prácticas de la industria.

1.1 Contexto y motivación

El proyecto surge como ejercicio integrador del bootcamp de desarrollo backend, donde el equipo debía demostrar dominio de:

- Diseño e implementación de APIs REST con Express
- Modelado de base de datos relacional con Sequelize y PostgreSQL
- Autenticación segura mediante JWT y gestión de sesiones
- Trabajo en equipo con división clara de responsabilidades
- Estructuración de proyectos siguiendo el patrón MVC

2. Tecnologías y Stack Técnico

Tecnología	Rol en el proyecto	Justificación
Node.js	Runtime de servidor	Ecosistema JS unificado, alto rendimiento I/O asíncrono
Express.js	Framework web	Minimalista, flexible, amplio soporte de middlewares
Sequelize ORM	Capa de acceso a datos	Abstracción SQL, migraciones, asociaciones declarativas
PostgreSQL	Base de datos relacional	Robustez, soporte de relaciones complejas, transacciones ACID
JSON Web Token (JWT)	Autenticación sin estado	Escalabilidad, sin sesiones en servidor, estándar del sector
Express-session	Gestión de sesiones web	Complementa JWT para vistas renderizadas en servidor
Pug (Jade)	Motor de plantillas	Integración nativa con Express, sintaxis concisa
bcrypt	Hash de contraseñas	Algoritmo estándar con sal para almacenamiento seguro
dotenv	Variables de entorno	Separación de configuración y código fuente
Postman	Pruebas de API	Colecciones reutilizables, documentación interactiva

2.1 Decisiones técnicas sobre el stack

¿Por qué PostgreSQL sobre MySQL o MongoDB?

Dado que el modelo de datos incluye relaciones complejas (usuarios → pokémon → equipos → pokémon de equipo), se optó por una base de datos relacional. PostgreSQL fue elegido frente a MySQL por su mayor cumplimiento del estándar SQL y su mejor soporte de tipos de datos avanzados. MongoDB fue descartado porque los datos del proyecto tienen estructura bien definida y relaciones múltiples que se benefician del esquema fijo.

¿Por qué Sequelize frente a SQL nativo o Prisma?

Sequelize permite definir modelos como clases JavaScript y gestionar las relaciones mediante métodos declarativos (hasMany, belongsTo, etc.), lo que acelera el desarrollo y reduce errores en queries complejas. Prisma fue considerado pero descartado al ser más adecuado para proyectos TypeScript; el equipo trabajó en JavaScript puro.

¿Por qué JWT + sesiones en paralelo?

La aplicación tiene dos superficies: una API REST (consumida por clientes externos o Postman) y vistas renderizadas en servidor (Pug). JWT resulta ideal para la API por ser stateless y portable. Las sesiones de Express complementan las vistas HTML donde el mantenimiento del estado entre peticiones es necesario para proteger rutas sin enviar tokens en cada request del navegador.

3. Arquitectura del Sistema

El backend sigue el patrón MVC (Modelo - Vista - Controlador) con una capa de servicios intermedia, lo que resulta en una separación clara de responsabilidades:

Capa	Responsabilidad	Archivos clave
Rutas (Routes)	Definir endpoints y aplicar middlewares	routes/*.js
Controladores	Recibir la petición HTTP y devolver la respuesta	controllers/*.js
Servicios	Lógica de negocio, validaciones, operaciones complejas	services/*.js
Modelos	Definición del esquema y queries a la base de datos	models/*.js
Middlewares	Autenticación JWT, control de sesión y roles	middlewares/*.js
Vistas	Plantillas Pug renderizadas en servidor	views/*.pug

3.1 Estructura de directorios

```
DOCS/
├── diagrams/
│   ├── flujo_admin.svg
│   ├── flujo_normal.svg
│   └── tablas.png
├── Memoria.pdf
└── README.md
public/
├── fonts/
│   └── pokemon-solid.ttf
├── style.css
├── admin.css
├── dashborad.css
├── index.css
├── login-register.css
├── fpokemon.css
├── store.css
└── teams.css
src/
├── config/
│   └── db.js # Conexión Sequelize + PostgreSQL
├── controllers/ # Reciben petición → llaman servicio → responden
│   ├── adminController.js
│   ├── authController.js
│   ├── pokemonController.js
│   ├── productsController.js
│   ├── teamController.js
│   ├── userController.js
│   ├── userPokemonController.js
│   └── userStoreController.js
├── middlewares/
│   ├── authMiddleware.js # Verificación JWT (Authorization: Bearer)
│   └── sessionMiddleware.js # Sesión + control de roles (user / admin)
└── models/
```

```

├── index.js                # Carga modelos y define asociaciones globales
├── pokemonModel.js
├── productsModel.js
├── teamModel.js
├── teamPokemonModel.js
├── userModel.js
├── userPokemonModel.js
├── userStoreController.js
├── routes/
│   ├── index.js           # Router raíz - monta todos los sub-routers
│   ├── adminRoutes.js
│   ├── authRoutes.js
│   ├── pokemonRoutes.js
│   ├── productsRoutes.js
│   ├── teamRoutes.js
│   ├── userPokemonRoutes.js
│   ├── userRoutes.js
│   ├── userStoreRoutes.js
│   └── viewRoutes.js
├── services/              # Lógica de negocio desacoplada
│   ├── adminService.js
│   ├── authService.js
│   ├── pokemonService.js
│   ├── productService.js
│   ├── teamService.js
│   ├── userPokemonService.js
│   ├── userService.js
│   └── userStoreService.js
├── views/                 # Plantillas Pug (server-side rendering)
│   ├── admin.pug
│   ├── dashboard.pug
│   ├── error.pug
│   ├── index.pug
│   ├── layout.pug
│   ├── login.pug
│   ├── pokemon.pug
│   ├── register.pug
│   ├── store.pug
│   └── teams.pug
└── index.js              # Entry point: Express, middlewares globales, sync DB
  
```

3.2 Decisión: por qué separar Servicios de Controladores

Una decisión clave fue introducir la capa de servicios en lugar de poner la lógica de negocio directamente en los controladores. Esto permite:

- Reutilización: un servicio puede ser llamado desde distintos controladores
- Testabilidad: los servicios son funciones puras, fáciles de testear en aislamiento
- Mantenibilidad: cambios de lógica no implican tocar la capa HTTP
- Claridad: los controladores quedan como adaptadores finos entre HTTP y negocio

4. Modelo de Datos y Relaciones

El esquema relacional del proyecto consta de seis entidades principales conectadas mediante relaciones de uno a muchos y muchos a uno:

Relación	Tipo	Significado
User → UserPokemon	One-to-Many	Un usuario puede capturar múltiples Pokémon
UserPokemon → User	Many-to-One	Cada Pokémon capturado pertenece a un usuario
UserPokemon → Pokemon	Many-to-One	Cada instancia referencia al Pokémon base (catálogo)
User → Team	One-to-Many	Un usuario puede tener múltiples equipos
Team → User	Many-to-One	Cada equipo pertenece a un usuario
Team → TeamPokemon	One-to-Many	Un equipo puede tener hasta 6 miembros
TeamPokemon → Team	Many-to-One	Cada slot de equipo pertenece a un equipo
TeamPokemon → UserPokemon	Many-to-One	El slot referencia el Pokémon específico del usuario

4.1 Decisión de diseño: tabla intermedia TeamPokemon

Se utilizó una tabla de unión explícita (TeamPokemon) en lugar de una relación directa Team → UserPokemon. Esto permite:

- Añadir metadatos por slot (posición en el equipo, orden, estadísticas modificadas)
- Mantener integridad referencial: si un Pokémon es liberado, el slot puede gestionarse independientemente
- Facilitar futuras mecánicas de juego donde el slot importa (p.ej. posición en combate)

4.2 Tabla Pokemon como catálogo centralizado

El modelo Pokemon actúa como catálogo de datos base (tipo, estadísticas, imagen, descripción). La tabla UserPokemon representa la instancia concreta de un Pokémon capturado por un usuario, con posibilidad de añadir datos propios (mote, nivel, experiencia) sin duplicar el catálogo. Esta separación sigue el principio de normalización y evita redundancia.

4.3 Sincronización de tablas

Sequelize sincroniza el esquema automáticamente al arrancar con `sync({ alter: true })` en desarrollo, lo que permite iterar rápidamente sin migraciones manuales. Para producción, se recomienda migrar a `sequelize-cli` con migraciones versionadas.

5. API REST — Endpoints y Autenticación

5.1 Autenticación y autorización

El sistema implementa dos niveles de protección:

- JWT (JSON Web Token): para endpoints de la API REST. El token se genera al hacer login y se envía en la cabecera Authorization: Bearer <token>. El middleware authMiddleware.js verifica la firma y extrae el payload.
- Express-session + roles: para las vistas renderizadas. El middleware sessionMiddleware.js comprueba si el usuario tiene sesión activa y, para rutas de administrador, verifica el campo role del usuario en base de datos.

Flujo de autenticación JWT

```
POST /api/auth/register → crea usuario con contraseña hasheada (bcrypt)
POST /api/auth/login → verifica credenciales → firma JWT → devuelve token
GET /api/users → [authMiddleware] verifica JWT → acceso si válido
→ [roleCheck] deniega si role !== 'admin'
```

5.2 Tabla de endpoints de la API

Método	Ruta	Descripción	Protección
POST	/api/auth/register	Registro de usuario	Pública
POST	/api/auth/login	Login → devuelve JWT	Pública
GET	/api/pokemon	Listar todos los Pokémon	Pública
GET	/api/pokemon/:id	Detalle de un Pokémon	Pública
POST	/api/pokemon	Crear Pokémon en catálogo	Admin JWT
PATCH	/api/pokemon/:id	Editar Pokémon	Admin JWT
DELETE	/api/pokemon/:id	Eliminar Pokémon	Admin JWT
GET	/api/pokemon/mis-pokemons	Pokémon del usuario autenticado	JWT
GET	/api/products	Listar productos de la tienda	Pública
POST	/api/products	Crear producto	Admin JWT
PATCH	/api/products/:id	Editar producto	Admin JWT
DELETE	/api/products/:id	Eliminar producto	Admin JWT
GET	/api/users	Listar todos los usuarios	Admin JWT
PATCH	/api/users/:id	Editar perfil de usuario	JWT propio
DELETE	/api/users/:id	Eliminar usuario	Admin JWT

5.3 Vistas renderizadas (Pug)

Ruta	Vista	Protección
GET /	index.pug — Home	Pública
GET /pokemon	pokemon.pug — Pokédex	Pública
GET /products	store.pug — Tienda	Pública
GET /users/perfil	dashboard.pug — Mi perfil	Sesión activa
GET /users/admin	dashboard.pug — Panel admin	Rol admin
GET /:user_id	pokemon.pug — Pokémon de usuario	Sesión activa

6. Retos Técnicos y Soluciones Adoptadas

Reto 1 — Coexistencia de JWT y sesiones

Problema: la aplicación necesitaba proteger tanto endpoints REST (idealmente sin estado, con JWT) como rutas de vistas HTML (donde el navegador no gestiona tokens fácilmente).

Solución: se crearon dos middlewares independientes. `authMiddleware.js` intercepta el header `Authorization` para la API. `sessionMiddleware.js` comprueba `req.session.user` para las vistas. Ambos coexisten sin interferir gracias a que se aplican selectivamente en los routers correspondientes.

Reto 2 — Relaciones circulares en los modelos Sequelize

Problema: al definir las asociaciones en archivos separados (`pokemonModel.js`, `userPokemonModel.js`, etc.), surgían errores de referencia circular al requerir los modelos entre sí.

Solución: se centralizó toda la definición de asociaciones en `models/index.js`. Este archivo importa todos los modelos, los registra en Sequelize y luego llama a los métodos de asociación en orden seguro, eliminando las dependencias circulares.

Reto 3 — Seguridad en el almacenamiento de contraseñas

Problema: almacenar contraseñas en texto plano es una vulnerabilidad crítica.

Solución: se implementó `bcrypt` con un factor de coste de 10 rondas de sal. El hash se genera antes de guardar el usuario y se compara con `bcrypt.compare()` en el login. La contraseña original nunca se almacena ni se loguea en ningún punto del flujo.

Reto 4 — Control de roles sin sobrecomplicar el sistema

Problema: distinguir usuarios normales de administradores sin añadir sistemas de permisos complejos.

Solución: el modelo User incluye un campo role con valores 'user' y 'admin'. El middleware de sesión comprueba este campo y rechaza el acceso si no coincide con el rol requerido. Es un sistema simple, suficiente para el alcance del proyecto y fácilmente extensible en el futuro.

Reto 5 — Sincronización del esquema en desarrollo ágil

Problema: en las fases iniciales, el esquema de base de datos cambiaba frecuentemente con cada nueva feature o corrección de diseño, requiriendo borrar y recrear tablas manualmente.

Solución: se configuró Sequelize con `sync({ alter: true })` en el entorno de desarrollo. Esto aplica automáticamente las diferencias entre el modelo definido en código y la tabla existente en PostgreSQL, sin borrar datos. Para producción se documenta la necesidad de migrar a migraciones controladas con sequelize-cli.

Reto 6 — Coordinación de equipo sin conflictos de merge

Problema: cuatro desarrolladores trabajando en paralelo sobre la misma base de código generaba conflictos frecuentes en Git, especialmente en `index.js` y `models/index.js`.

Solución: se acordó una arquitectura de rutas modular donde cada módulo (auth, pokemon, products, team, users) vive en su propio archivo de rutas y controlador. El router raíz (`routes/index.js`) simplemente monta cada sub-router. Esto redujo drásticamente los conflictos al concentrar cada módulo en su rama de trabajo.

7. Reparto de Tareas y Contribuciones

7.1 División de responsabilidades

El equipo se organizó asignando módulos completos verticales (modelo + servicio + controlador + rutas) a cada miembro, evitando dependencias cruzadas durante el desarrollo paralelo.

Módulo	Responsable	Archivos principales
User + Auth + Relaciones	Asier Gonzales	userModel.js, userService.js, userController.js, authController.js, authMiddleware.js, sessionMiddleware.js, models/index.js
Pokemon + Documentación	Darío Arenaza	pokemonModel.js, pokemonService.js, pokemonRoutes.js, userPokemonModel.js, userPokemonController.js, README.md
Product + Vistas	Luis Alonso	productModel.js, productService.js, productsController.js, productsRoutes.js, views/*.pug, viewRoutes.js
Team + TeamPokemon	Eli Fernández	teamModel.js, teamPokemonModel.js, teamService.js, teamController.js, teamRoutes.js

7.2 Detalle por miembro

Asier Gonzales — User, Auth y Relaciones

Asier tomó el rol de arquitecto de seguridad y datos del proyecto. Sus responsabilidades incluyeron:

- Diseño del modelo User con campos email, password (hashed), username, role, coins
- Implementación del flujo completo de registro con validación de email único y hash bcrypt
- Generación y verificación de JWT en el login, con payload mínimo (id, role)
- Middleware de autenticación JWT (authMiddleware.js) aplicable a cualquier ruta
- Middleware de sesión con control de roles (sessionMiddleware.js)
- Definición de todas las asociaciones Sequelize en models/index.js
- Configuración de la conexión a PostgreSQL (config/db.js) con dotenv

Darío Arenaza — Pokemon y Documentación

Darío gestionó el catálogo de Pokémon y la documentación técnica del proyecto:

- Modelo Pokemon con campos name, type, stats (HP, ataque, defensa), imagen, descripción
- Modelo UserPokemon como tabla de Pokémon capturados por usuario (instancias)
- Servicio y controlador para operaciones CRUD del catálogo (restringidas a admin)
- Endpoint GET /api/pokemon/mis-pokemons para listar los Pokémon del usuario autenticado
- Redacción del README.md con instrucciones de instalación, rutas y credenciales de prueba
- Documentación de la colección Postman con ejemplos de cada endpoint

Luis Alonso — Products y Vistas

Luis desarrolló el sistema de tienda y las vistas renderizadas en servidor:

- Modelo Product con campos name, description, price, type (objeto, pokéball, etc.), stock
- CRUD completo de productos con restricciones de rol admin en creación, edición y borrado
- Integración de la lógica de compra: descuento de coins del usuario al adquirir un producto
- Layout base en Pug (layout.pug) con navbar, header y bloques extensibles
- Vistas: home, login, registro, pokédex, tienda, dashboard de usuario y panel admin
- viewRoutes.js: rutas que renderizan las vistas pasando datos del servicio a las plantillas

Eli Fernández — Team, TeamPokemon y Diagramas

Eli diseñó e implementó el sistema de equipos, la funcionalidad más compleja en términos de relaciones:

- Modelo Team con campos name, user_id, descripción, fecha de creación
- Modelo TeamPokemon como tabla de unión entre Team y UserPokemon con campo position
- Servicio teamService.js con lógica de negocio: validar que el Pokémon pertenece al usuario, límite de 6 miembros por equipo, unicidad de Pokémon en equipo
- Controlador y rutas para crear equipo, añadir/eliminar miembros, listar equipos del usuario
- Vista teams.pug que muestra los equipos del usuario con sus miembros y permite gestión

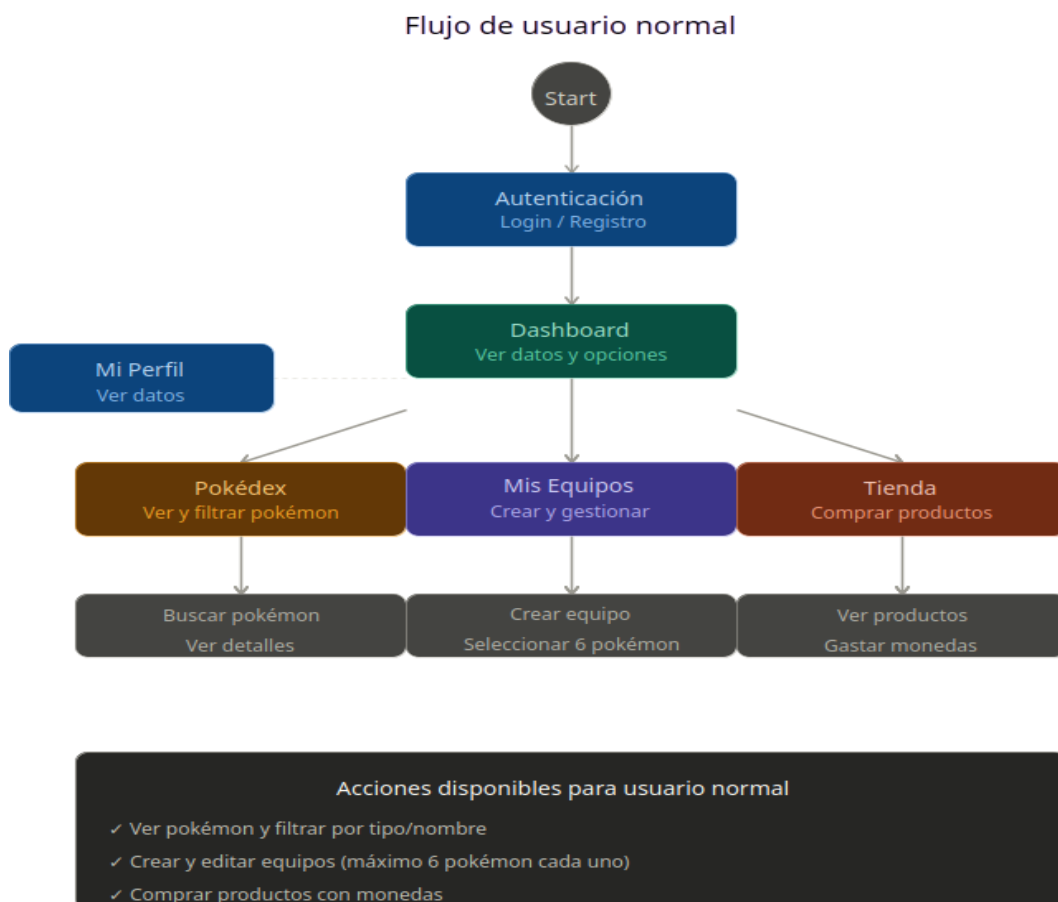
8. Flujo de Datos — Ejemplo de Petición

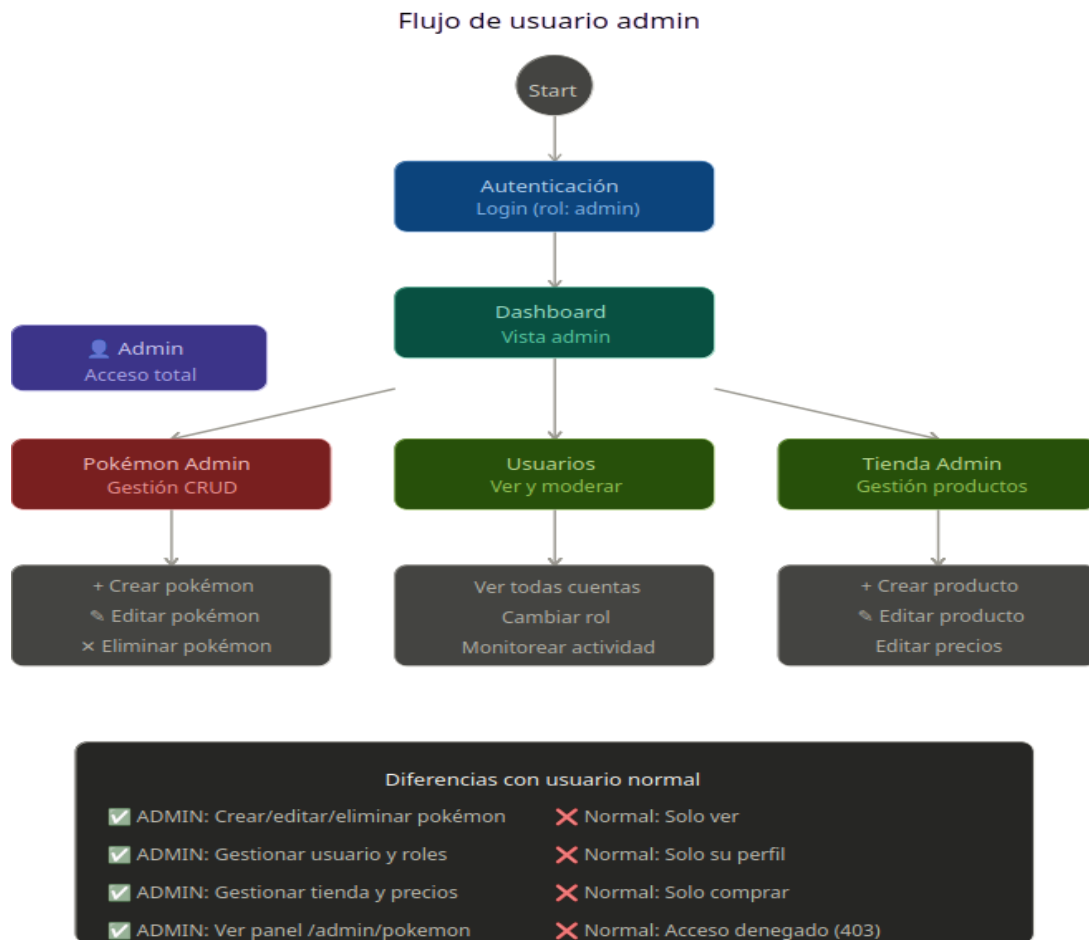
A continuación se detalla el ciclo de vida completo de una petición autenticada para ilustrar cómo interactúan todas las capas:

Ejemplo: POST /api/products/:id/buy (comprar un producto)

1. El cliente envía POST /api/products/3/buy con cabecera Authorization: Bearer <jwt>
2. El router products aplica authMiddleware.js antes de llegar al controlador
3. authMiddleware verifica la firma del JWT y adjunta el payload (id, role) a req.user
4. productsController.js llama a productService.buyProduct(userId, productId)
5. productService verifica que el producto existe y tiene stock disponible
6. productService comprueba que el usuario tiene suficientes coins (consulta userService)
7. Sequelize ejecuta una transacción: descuenta coins del usuario y reduce stock del producto
8. Si la transacción falla (p.ej. stock a 0 entre verificación y compra), hace rollback
9. El servicio devuelve el resultado al controlador
10. El controlador responde 200 OK con el producto comprado o 400/409 con mensaje de error

Este flujo demuestra la separación de responsabilidades: el controlador no conoce la lógica de negocio, el servicio no conoce HTTP, y el modelo no conoce ninguno de los dos.





9. Seguridad

Medida	Implementación	Protege contra
Hash de contraseñas	bcrypt con 10 rondas de sal	Filtración de base de datos
JWT firmado	HS256 con secret en .env	Suplantación de identidad
Variables de entorno	.env excluido de Git (.gitignore)	Exposición de credenciales
Control de roles	Middleware comprueba role en BD	Escalada de privilegios
Validación de pertenencia	El servicio verifica owner antes de operar	Acceso a recursos ajenos
Transacciones SQL	Sequelize transaction() en operaciones críticas	Inconsistencia de datos

10. Instalación y Configuración

10.1 Requisitos previos

- Node.js v18 o superior
- PostgreSQL 14 o superior
- npm 9 o superior

10.2 Pasos de instalación

```
# 1. Clonar el repositorio
git clone <repo-url>
cd pokeback

# 2. Instalar dependencias
npm install

# 3. Crear el archivo de entorno
cp .env.example .env
# Editar .env con los datos de PostgreSQL:
#   DB_HOST=localhost
#   DB_PORT=5432
#   DB_NAME=pokeback_db
#   DB_USER=postgres
#   DB_PASSWORD=tu_contraseña
#   JWT_SECRET=tu_secreto_seguro
#   SESSION_SECRET=otro_secreto

# 4. Crear la base de datos en PostgreSQL
psql -U postgres -c 'CREATE DATABASE pokeback_db;'

# 5. Arrancar el servidor (sincroniza tablas automáticamente)
npm run dev
```

10.3 Credenciales de prueba

Email	Contraseña	Rol
luis@mail.com	1234	user

11. Líneas de Mejora Futuras

Mejora	Descripción	Prioridad
Migraciones versionadas	Sustituir <code>sync({ alter })</code> por <code>sequelize-cli migrations</code> para producción	Alta
Refresh tokens	Añadir refresh token con mayor TTL para mejorar UX de sesión	Alta
Tests automatizados	Suite Jest + Supertest para los endpoints principales	Alta
Sistema de combate PvE	Motor de batallas por turnos usando los equipos existentes	Media
WebSockets PvP	Combates en tiempo real entre usuarios con Socket.io	Media
Rate limiting	Limitar peticiones por IP con <code>express-rate-limit</code>	Alta
Documentación Swagger	Generar docs interactiva con <code>swagger-jsdoc</code> desde los JSDoc	Media
Cache de catálogo	Redis para cachear el catálogo de Pokémon (datos estáticos)	Baja

12. Conclusiones

Con el proyecto POKEback hemos cumplido con demostrar dominio del desarrollo backend con Node.js, Express, Sequelize y PostgreSQL, aplicando buenas prácticas de arquitectura y seguridad en un contexto de trabajo en equipo real.

Las decisiones técnicas tomadas (separación de capas MVC + servicios, doble sistema de autenticación, centralización de asociaciones, transacciones SQL) han resultado en una base sólida y extensible, sobre la que se pueden implementar las futuras mecánicas de juego sin necesidad de refactorizaciones profundas.

El reparto de módulos verticales por miembro ha demostrado ser la estrategia más efectiva para trabajar en paralelo minimizando conflictos, aunque se identificó la necesidad de sincronizar más frecuentemente los cambios en archivos transversales como `models/index.js`.

Como aprendizajes clave del equipo destacamos: la importancia de definir el modelo de datos antes de empezar a codificar, el valor de los middlewares reutilizables para la autenticación, y que una arquitectura bien pensada desde el inicio ahorra tiempo en las fases avanzadas del proyecto.